Generalization Error

Fatemeh Mansoori

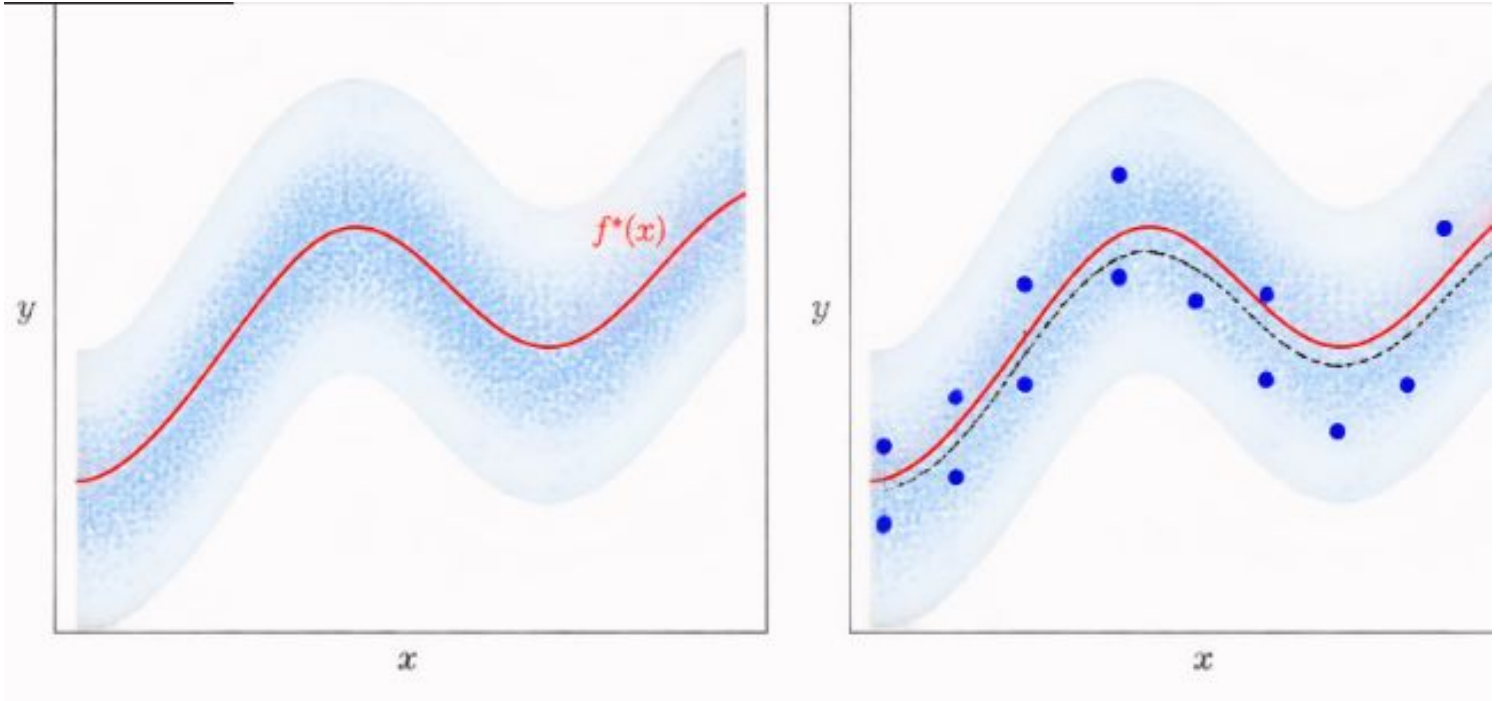
Evaluation and generalization

Why minimizing the cost function (based on only training data) while we are interested in the performance on new examples?

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} Loss\left(y^{(i)}, f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})\right) \longrightarrow \text{Empirical loss}$$

Evaluation: After training, we need to measure how well the learned prediction function can predicts the target for unseen examples

Distribution of data



Training and test performance

Assumption: training and test examples are drawn independently at random from the same but unknown distribution.

- Each training/test example $(\boldsymbol{x}, y)$ is a sample from joint probability distribution $P(\boldsymbol{x}, y)$, i.e., $(\boldsymbol{x}, y) \sim P$

Empirical (training) loss = $\frac{1}{n}\sum_{i=1}^{n} Loss\left(y^{(i)}, f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})\right)$

Expected (test) loss = $E_{x,y}\left\{Loss(y, f(\boldsymbol{x}; \boldsymbol{\theta}))\right\}$

Challenge of computing test (true) error

We don’t know the true distribution P(x,y)

We can not calculate the exact expected test error

How do we estimate it in practice ?

We use an independent test dataset and compute the error for this dataset

Under the assumption that test samples

- are independently drawn from the same distribution
- have not been used to train or select the model

test error provides an unbiased estimate of its generalization error.

Expected Test Error

Definition: Expected performance on unseen data

- Test data sampled from the same distribution p(x, y)
- Approximate using test set
- Generalization gap is the difference between training and test performance.
-

$$J_{\text{train}}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - f\left(x^{(i)}, w\right)\right)^2$$

$$J_{\text{test}}(w) = \frac{1}{m}\sum_{i=1}^{m}\left(y_{\text{test}}^{(i)} - f\left(x_{\text{test}}^{(i)}, w\right)\right)^2$$

$$\text{Gap}(w) = J_{\text{gen}}(w) - J_{\text{train}}(w).$$

Overfitting & Underfitting Definition

Overfitting :

The model fits the training data well but performs poorly on the test set

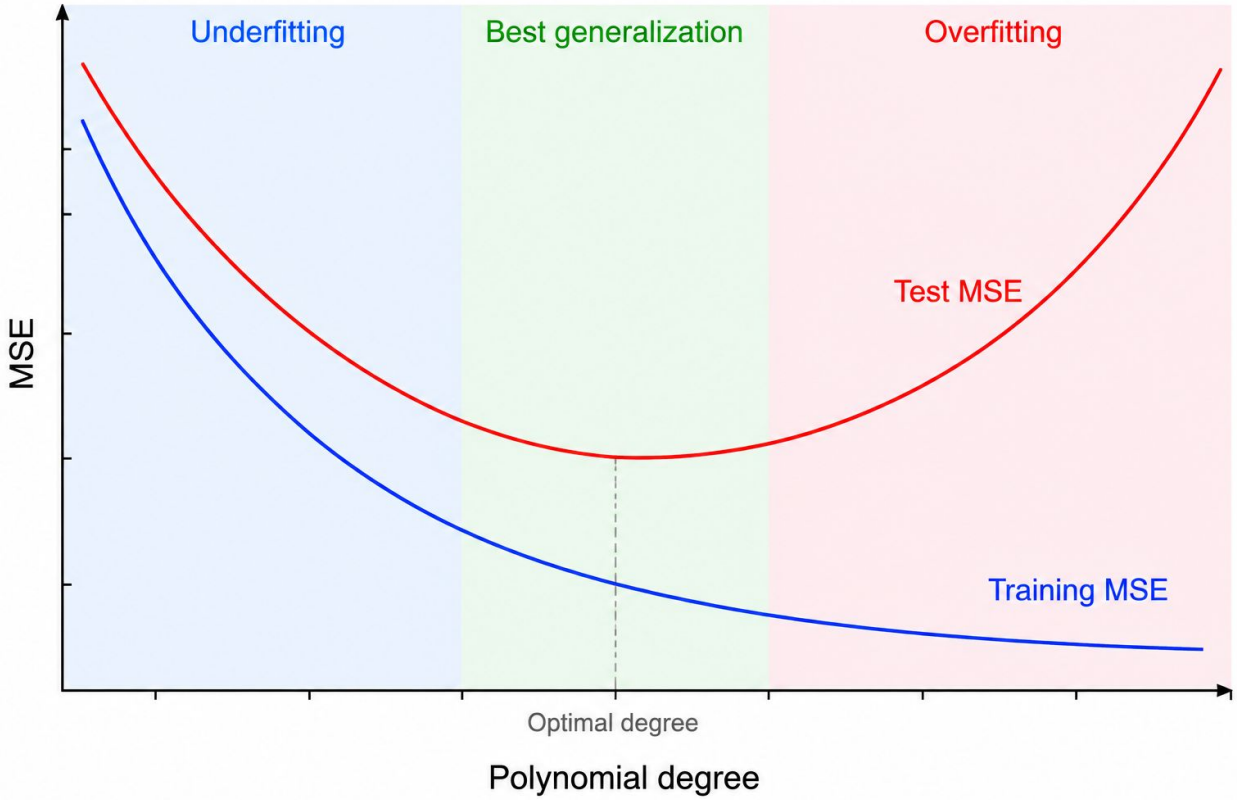
$$J_{\text{train}}(w) \ll J_{\text{test}}(w)$$

Underfitting:

The model is too simple and cannot capture the structure of the data

$$J_{\text{train}}(w) \approx J_{\text{test}}(w) \gg 0$$

Polynomial regression (train and test error)



Overfitting causes

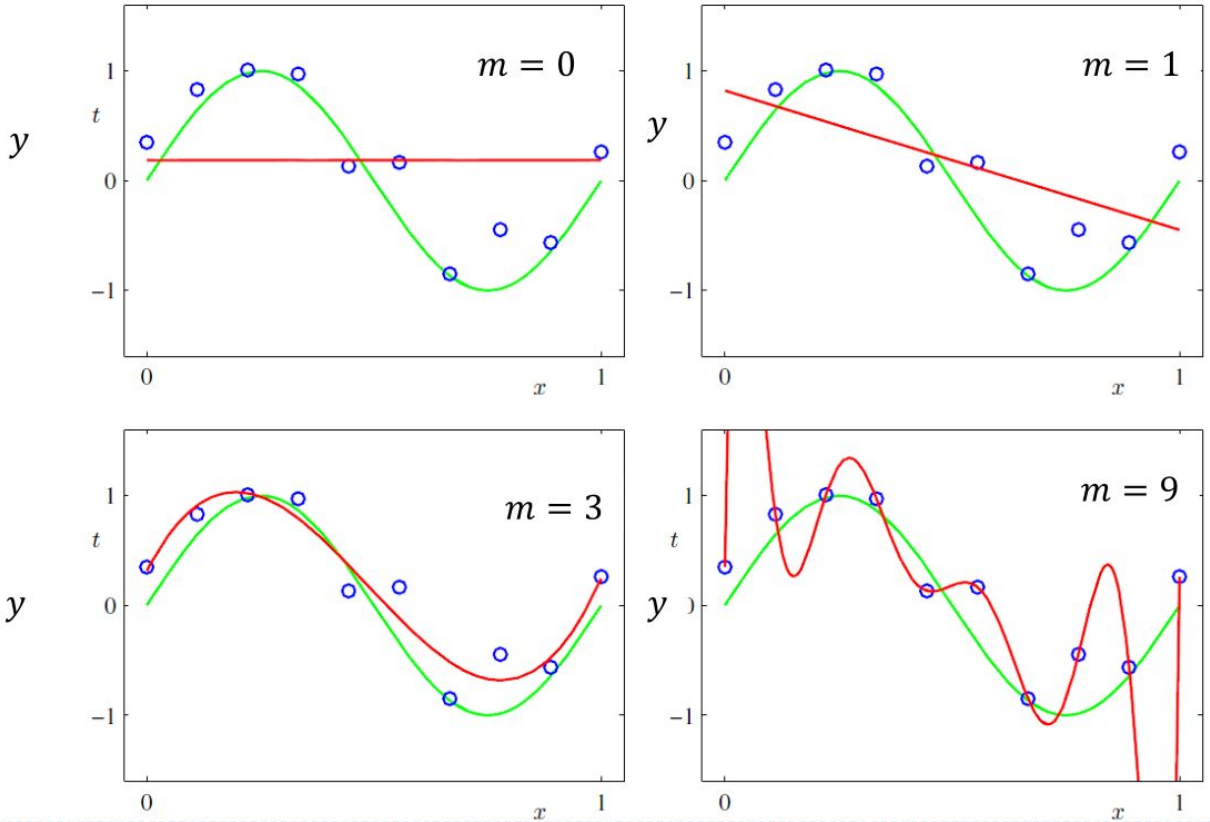
Model complexity

- E.g., Model with a large number of parameters (degrees of freedom)

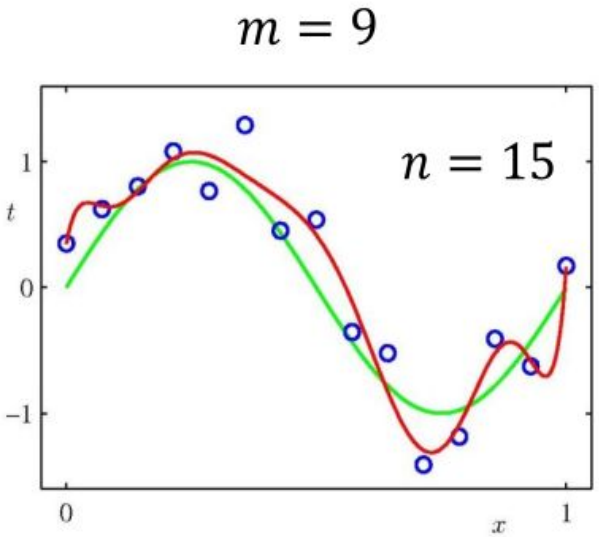
Low number of training data

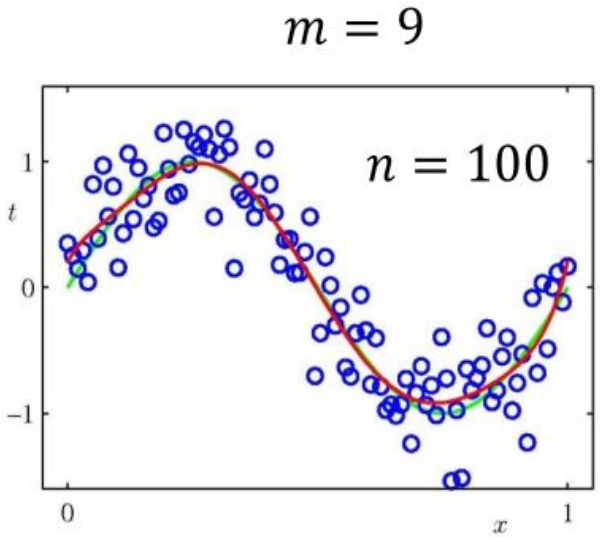
- Small data size compared to the complexity of the model

Polynomial regression (model complexity)

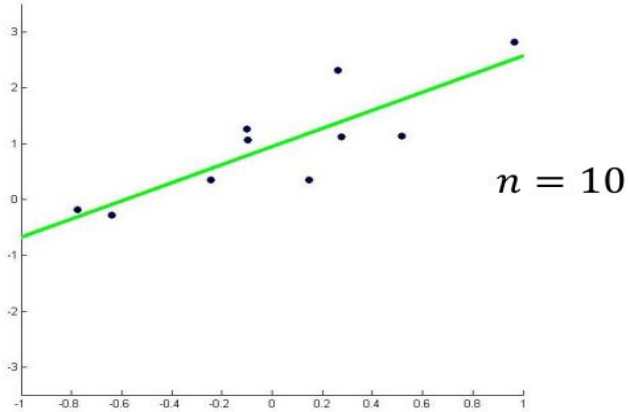


Number of training data and overfitting

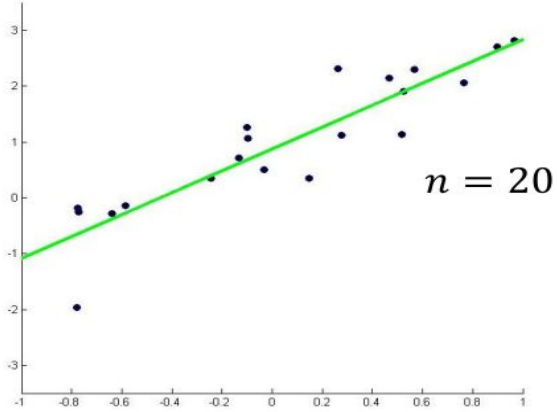




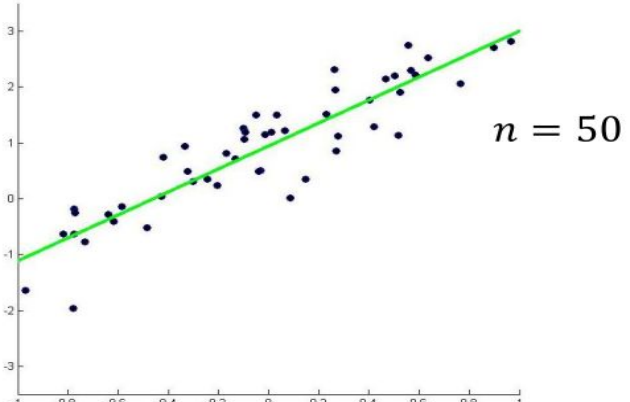
Number of training data



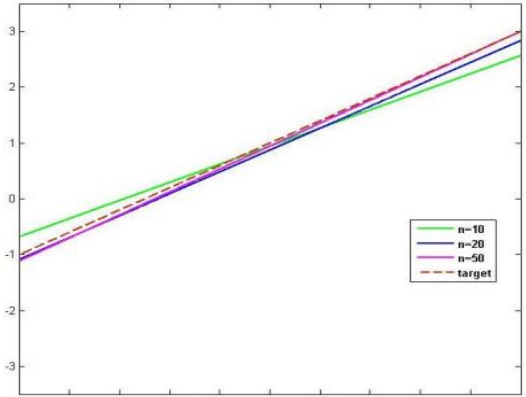
$n = 10$



$n = 20$



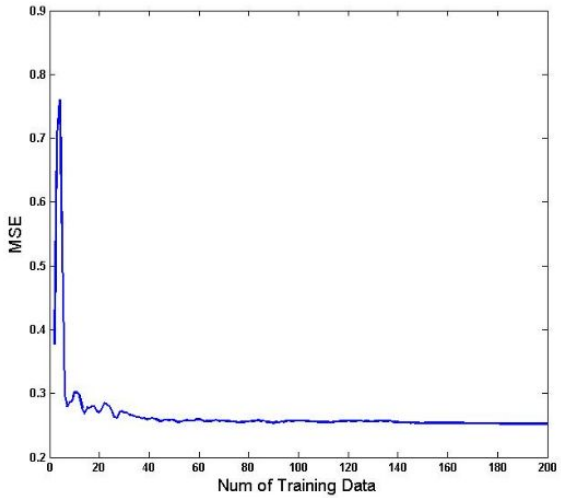
$n = 50$



Generalization

By increasing the number of training examples, will solution be better?

Why the mean squared error does not decrease more after reaching a level?

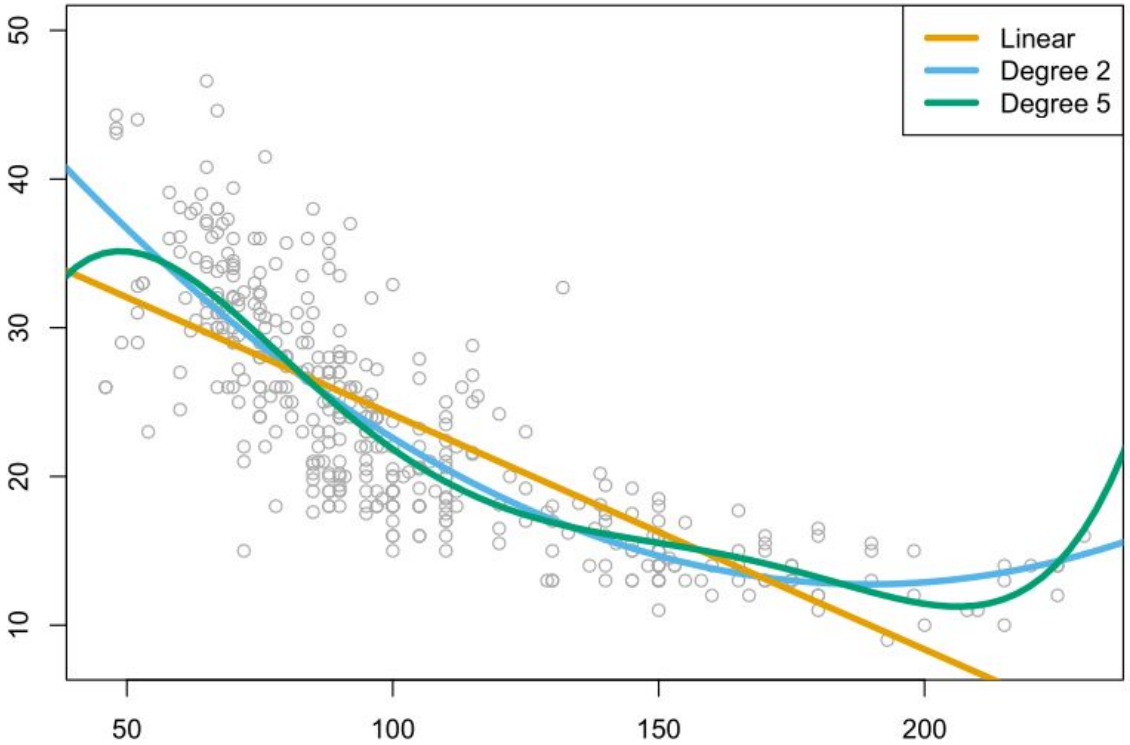


Evaluation and Model selection

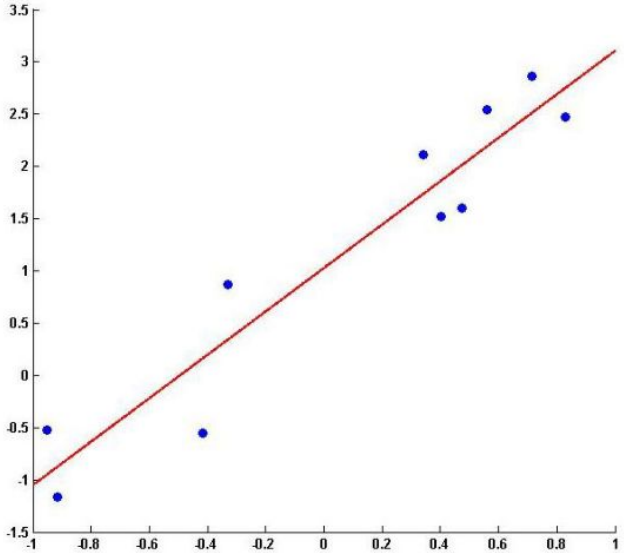
- **Evaluation**:
  - We need to measure how well the learned function can predicts the target for unseen examples

- **Model selection:**
  - Most of the time we need to select among a set of models
    - Example: polynomials with different degree m
  - and thus we need to evaluate these models first

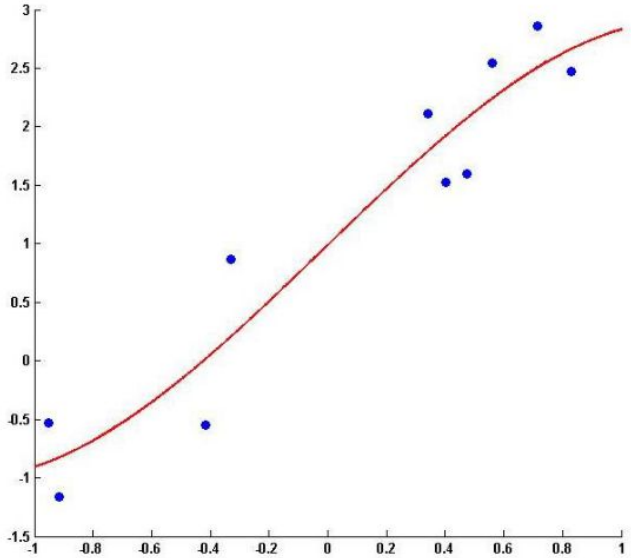
Using evaluation for Model selection



Model selection: polynomial regression

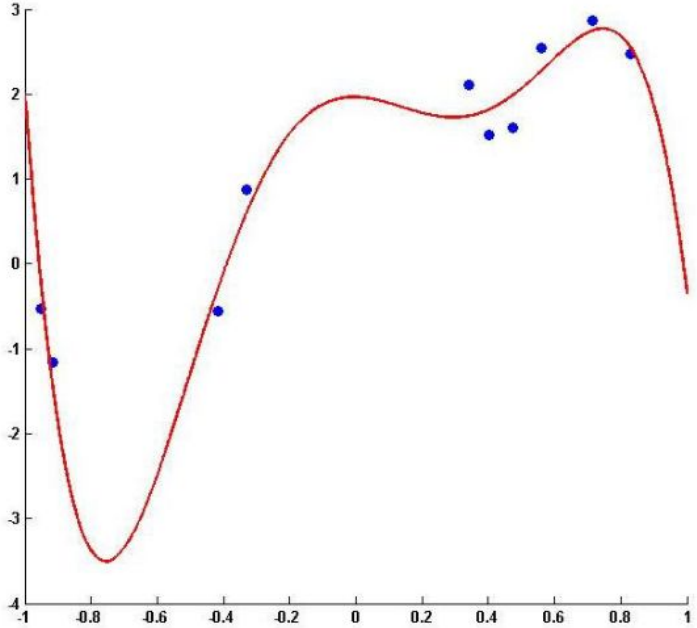


Degree of 1

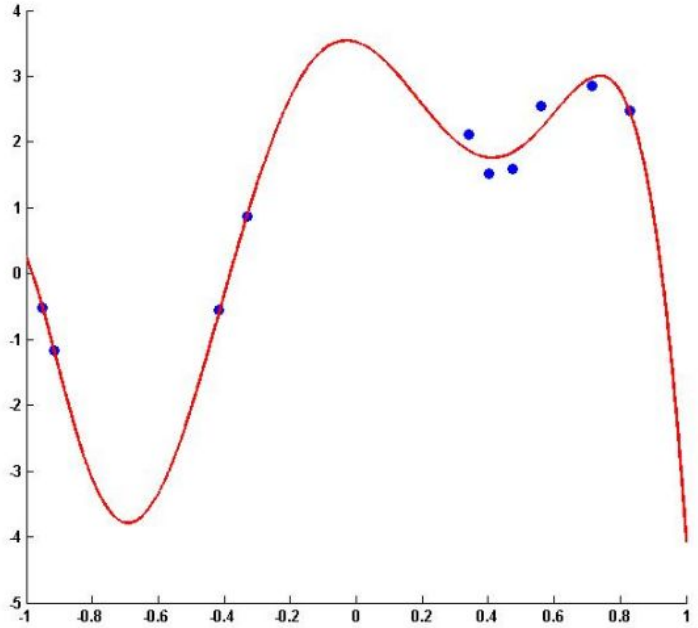


Degree of 3

Model selection: polynomial regression



Degree of 5



Degree of 7

Avoiding overfitting

Determine a suitable value for model complexity

- **Simple hold-out method**
- **Cross-validation**

Regularization (Occam's Razor)

- Explicit preference towards simple models
- Penalize for the model complexity in the objective function

Simple hold-out model selection

- Steps:
  - Divide training data into training and validation set v_set
  - Use only the training set to train a set of models
  - Evaluate each learned model on the validation set
    - $J_v(\boldsymbol{w}) = \frac{1}{|v_set|} \sum_{i \in v_set} \left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}; \boldsymbol{w} \right) \right)^2$
  - Choose the best model based on the validation set error

- Usually, too wasteful of valuable training data
  - Training data may be limited.
  - On the other hand, small validation set give a relatively noisy estimate of performance.

Simple hold-out

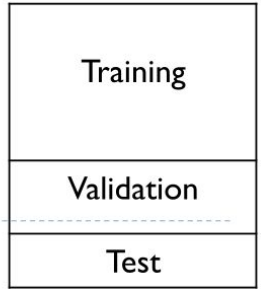
Simple hold-out chooses the model that minimizes error on validation set.

$J_v(\hat{\boldsymbol{w}})$ is likely to be an optimistic estimate of generalization error.

- extra parameter (e.g., degree of polynomial) is fit to this set.

Estimate generalization error for the test set

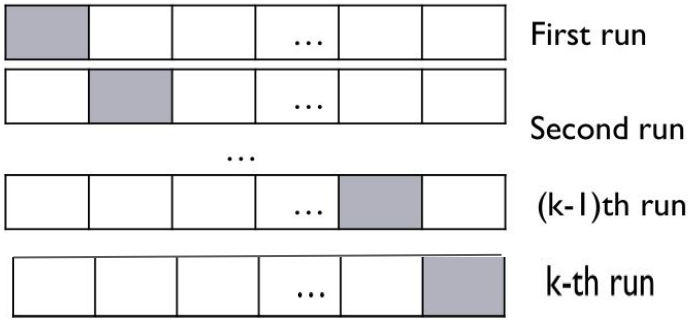
- performance of the selected model is finally evaluated on the test set



Cross validation (CV) : Evaluation

k-fold cross-validation steps:

- Shuffle the dataset and randomly partition training data into k groups o approximately equal size
- for $i = 1$ to k
  - Choose the i-th group as the held-out validation group
  - Train the model on all but the i-th group of data
  - Evaluate the model on the held-out group
- Performance scores of the model from k runs are **averaged**.
  - The average error rate can be considered as an estimation of the tru performance.



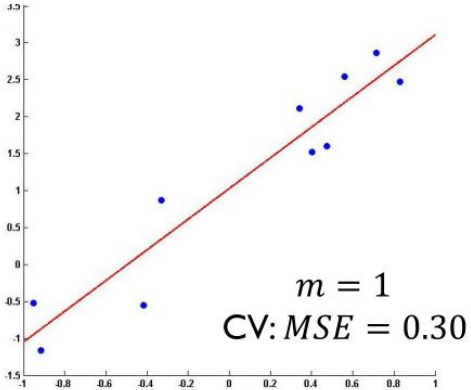
Cross validation (CV) : Evaluation

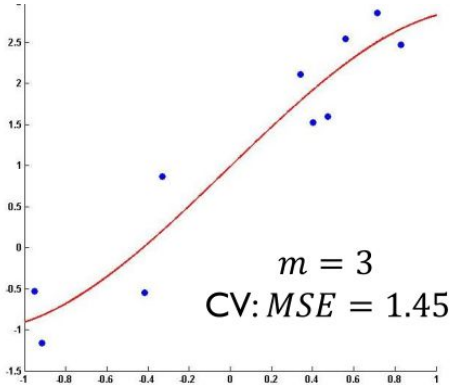
For each model we first find the average error find by CV.

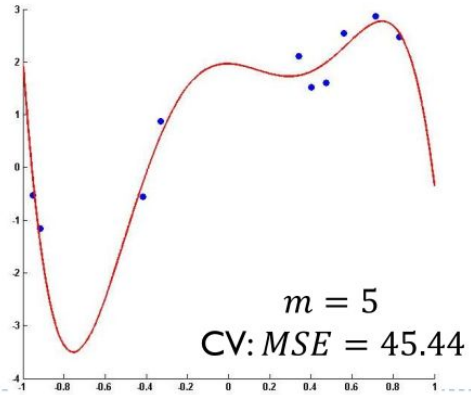
The model with **the best average performance** is selected.

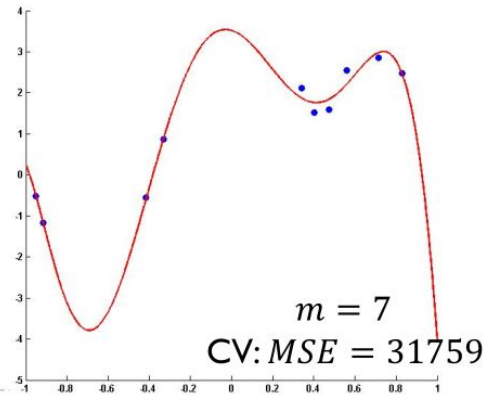
Cross-validation: polynomial regression example

- 5-fold CV
- 100 runs
  - average



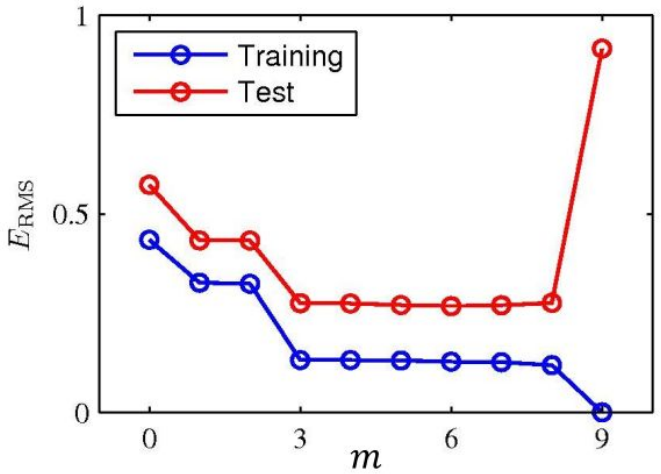






Polynomial regression: training and test error

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)};\boldsymbol{\theta}\right)\right)^2}{n}}$$



[Bishop]

Regularization

Purpose: Prevent overfitting by penalizing large weights

$$J_\lambda(w) = J(w) + \lambda R(w)$$

- Common regularizers: L1 and L2 norms
- λ controls the balance between fit and simplicity

Regularization

Polynomials with larger m are becoming increasingly tuned to the random noise on the target values.

- magnitude of the coefficients typically gets larger by increasing m.

	$M=0$	$M=1$	$M=6$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

[Bishop]

- Suppose $y(x) = w_0 + w_1 x + w_2 x^2$
- And two points (0,0), (0.1,1)
- Two possible solutions are :

$$w_1 = 10,\ w_2 = 0$$
$$w_1 = 0,\ w_2 = 100$$

- Large coefficients can also cancel one another, allowing the model to fit particular observations

Regularization

discourages the model from using unnecessarily large coefficients

Without regularization:

- model focuses only on reducing training error.
- It may use very large coefficients to fit random noise.

With regularization:

- large coefficients become costly.
- model must balance fitting the training data against keeping its coefficients under control.

Regularization

$$\lambda \sum_{j=1}^{M} w_j^2$$

Rigid or L2 Regularization :

$$J(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - f(x_n, \mathbf{w})\right)^2 + \lambda \mathbf{w}^T \mathbf{w}$$

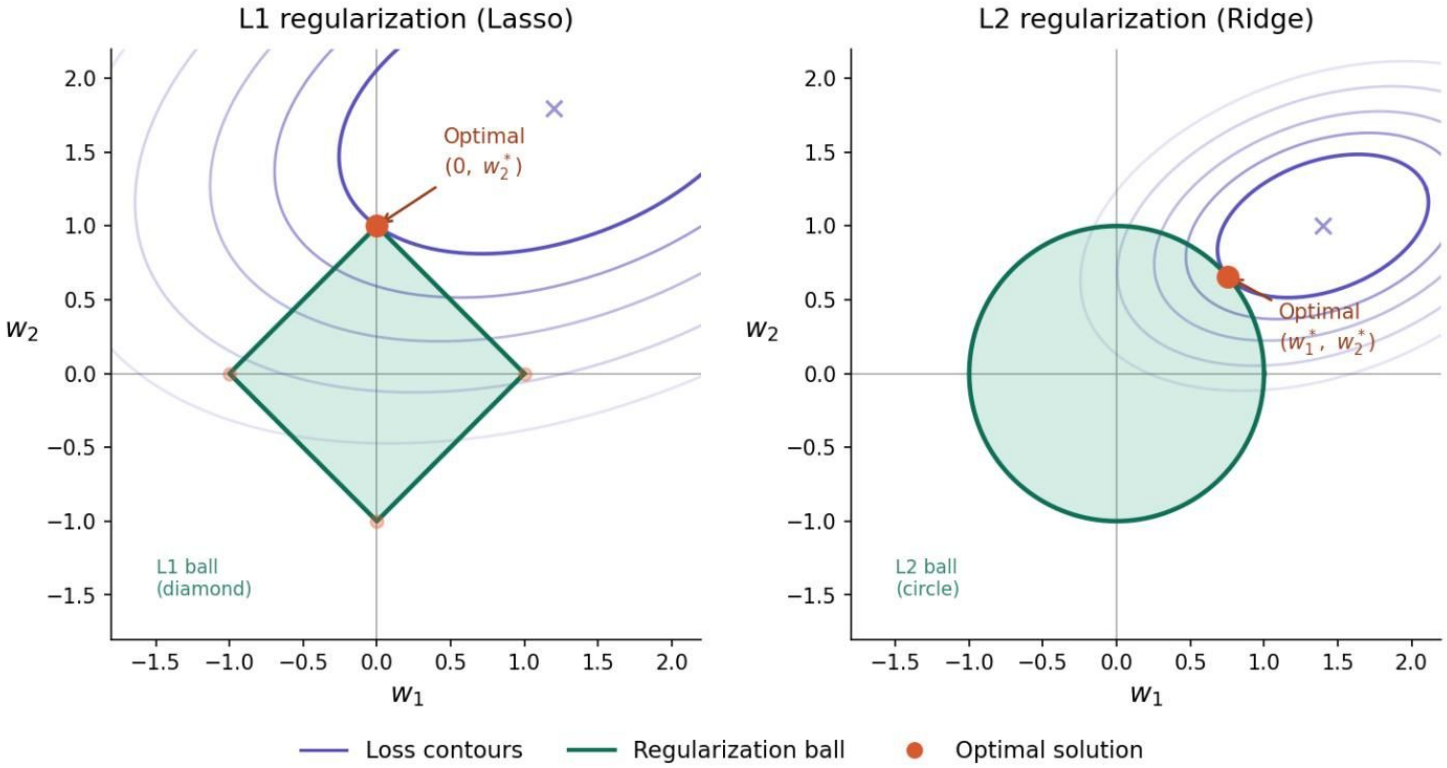
Lasso or L1 Regularization:

$$J(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - f(x_n, \mathbf{w})\right)^2 + \lambda \sum_{j=1}^{M} |w_j|$$

Lasso can produce sparse model and selecting features

Ridge tends to perform better when many features are useful, especially when the features are highly correlated (lasso keep one feature, rigid distribute weight among correlated features)

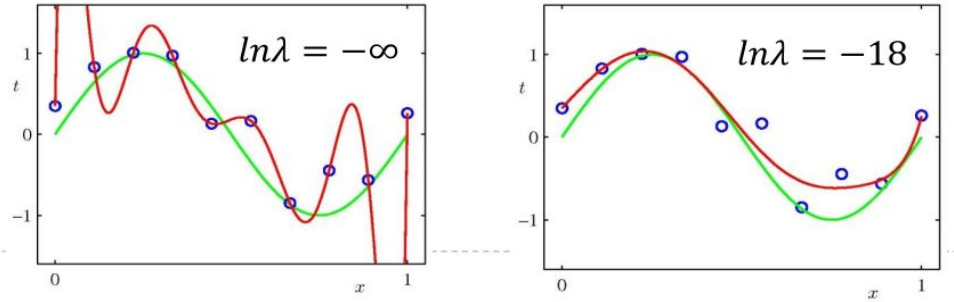
Rigid Vs Lasso



λ parameter

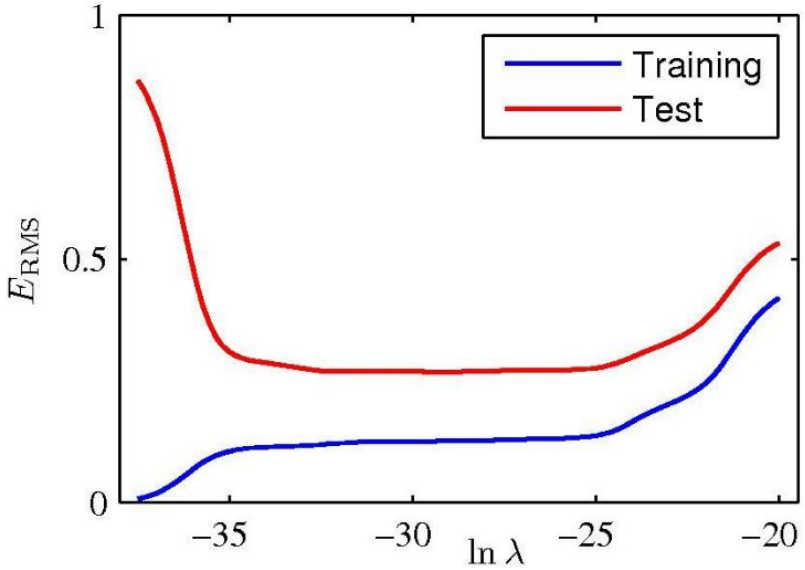
$m = 9$

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
$\widehat{w}_0$	0.35	0.35	0.13
$\widehat{w}_1$	232.37	4.74	-0.05
$\widehat{w}_2$	-5321.83	-0.77	-0.06
$\widehat{w}_3$	48568.31	-31.97	-0.05
$\widehat{w}_4$	-231639.30	-3.89	-0.03
$\widehat{w}_5$	640042.26	55.28	-0.02
$\widehat{w}_6$	-1061800.52	41.32	-0.01
$\widehat{w}_7$	1042400.18	-45.95	-0.00
$\widehat{w}_8$	-557682.99	-91.53	0.00
$\widehat{w}_9$	125201.43	72.68	0.01



λ parameter

λ now controls the effective complexity of the model and hence determines the degree of over-fitting



[Bishop]

Choosing the regularization parameter

A set of models with different values of λ.

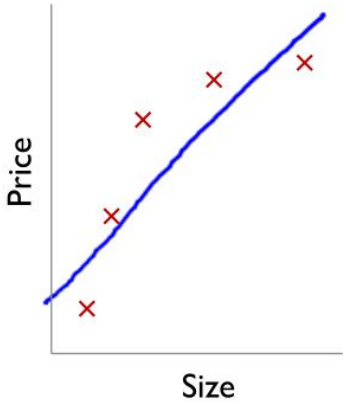
Find $\widehat{\boldsymbol{w}}$ for each model based on training data

Find $J_v(\widehat{\boldsymbol{w}})$ (or $J_{cv}(\widehat{\boldsymbol{w}})$) for each model

- $J_v(\boldsymbol{w}) = \frac{1}{n_v}\sum_{i \in v_set}\left(y^{(i)} - f\left(x^{(i)}; \boldsymbol{w}\right)\right)^2$

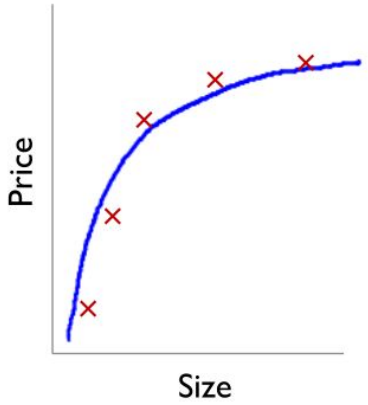
Select the model with the best $J_v(\widehat{\boldsymbol{w}})$ (or $J_{cv}(\widehat{\boldsymbol{w}})$)

Complexity of Hypothesis Space: Example

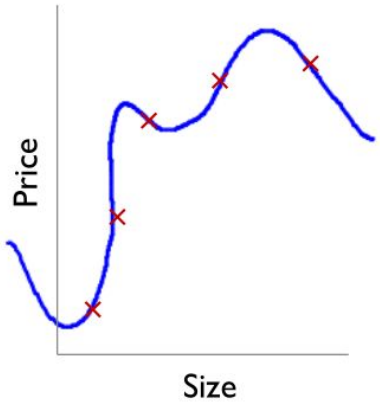


$w_0 + w_1 x$

Less complex $\mathcal{H}$



$w_0 + w_1 x + w_2 x^2$



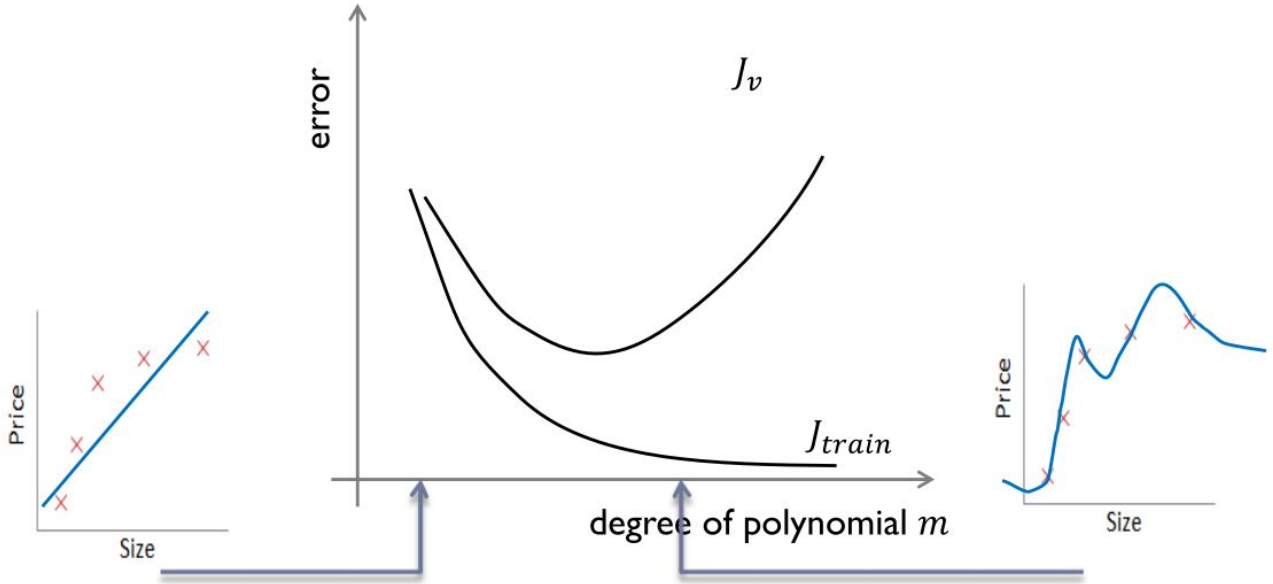
$w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$

More complex $\mathcal{H}$

Complexity of Hypothesis Space: Example

$$J_v(\boldsymbol{w}) = \frac{1}{n_v}\sum_{i \in val_set}\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}; \boldsymbol{w}\right)\right)^2$$

$$J_{train}(\boldsymbol{w}) = \frac{1}{n_train}\sum_{i \in train_set}\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}; \boldsymbol{w}\right)\right)^2$$

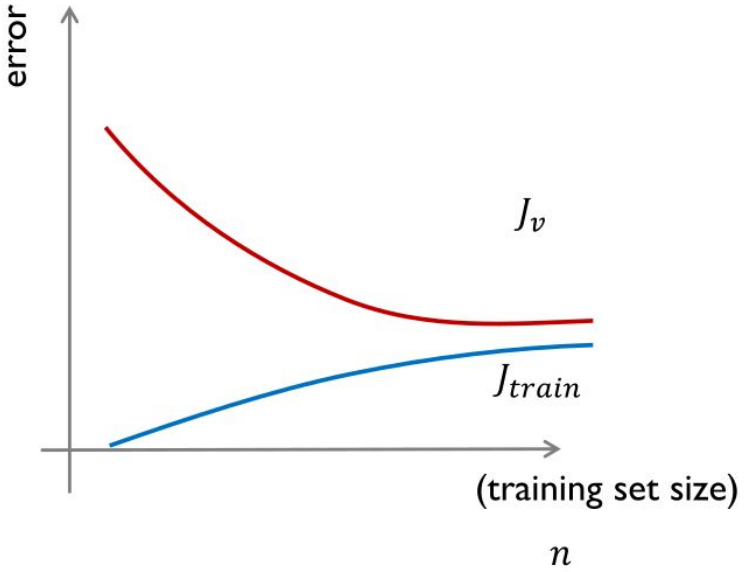


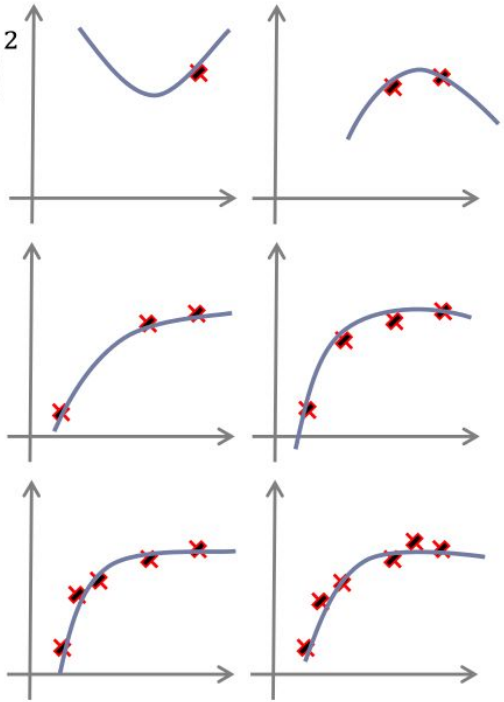
Size of training set

$$J_v(\boldsymbol{w}) = \frac{1}{n_v}\sum_{i \in val_set}\left(y^{(i)} - f\left(x^{(i)};\boldsymbol{w}\right)\right)^2$$

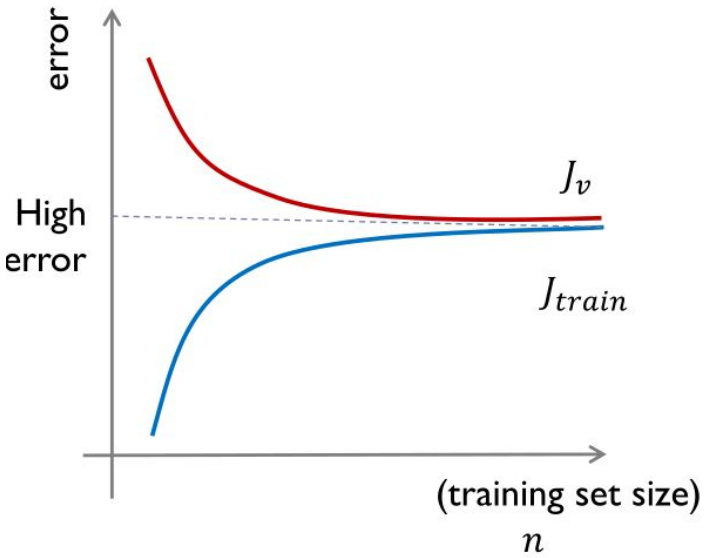
$$J_{train}(\boldsymbol{w}) = \frac{1}{n_train}\sum_{i \in train_set}\left(y^{(i)} - f\left(x^{(i)};\boldsymbol{w}\right)\right)^2$$

$$f(x;\boldsymbol{w}) = w_0 + w_1 x + w_2 x^2$$



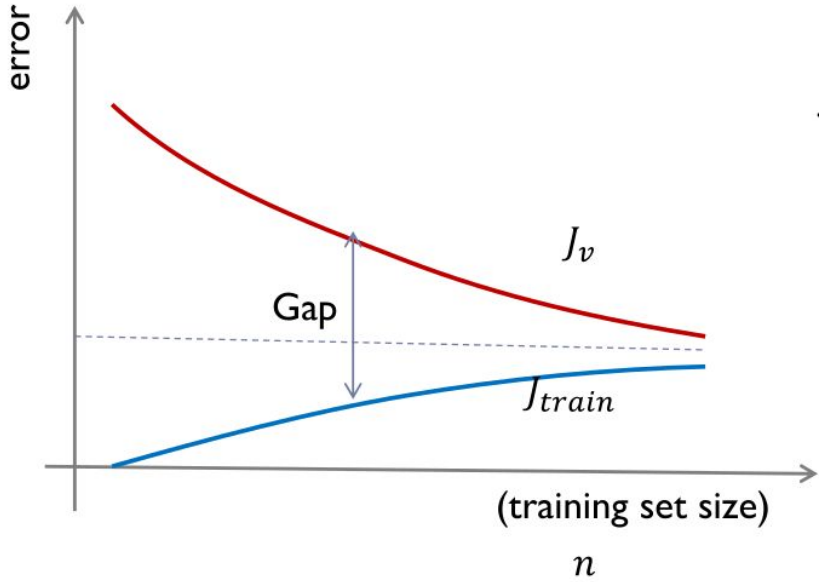


Less Complex H



If model is very simple, getting more training data will not (by itself) help much.

More complex H

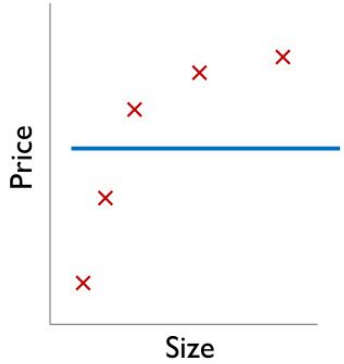


For more complex models, getting more training data is usually helps.

Regularization: Example

$$f(x; \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$

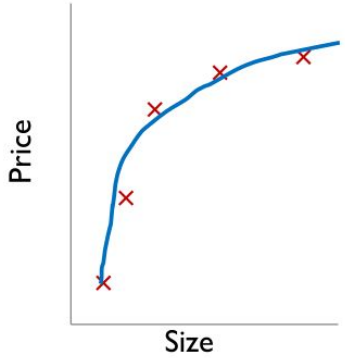
$$J(\boldsymbol{w}) = \frac{1}{n}\left(\sum_{i=1}^{n} \left(y^{(i)} - f\left(x^{(i)}; \boldsymbol{w}\right)\right)^2 + \lambda \boldsymbol{w}^T \boldsymbol{w}\right)$$



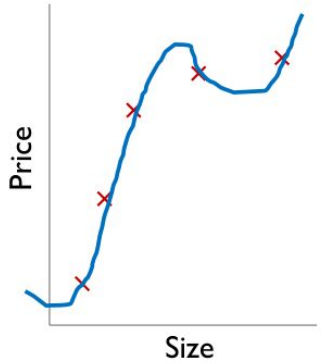
Large λ

(Prefer to more simple models)

$w_1 = w_2 \approx 0$



Intermediate λ



Small λ

(Prefer to more complex models)

$\lambda = 0$

Generalization error decomposition:

$$\mathbb{E}\left[(y - f(x, w))^2\right] = \text{Bias}(x)^2 + \text{Variance}(x) + \text{Noise}$$

Bias: Error due to simplifying assumptions in the model

$$\text{Bias}(x) = \mathbb{E}[f(x, w)] - f_{\text{true}}(x)$$

Variance: Sensitivity of the model to training data

$$\text{Variance}(x) = \mathbb{E}\left[(f(x, w) - \mathbb{E}[f(x, w)])^2\right]$$

Noise: Irreducible error from the inherent randomness in data

Bias-Variance Tradeoff

Tradeoff: Balancing between bias and variance is key for optimal performance

- Low complexity: High bias, low variance
- High complexity: Low bias, high variance

Reference

- M. Soleimani, *Machine Learning Course Lecture Slides*.
- Sharifi zarchi, Machine Learning Course Lecture Slides
- C. Bishop, “Pattern Recognition and Machine Learning”, Chapter 1.1,1.3
- A. Ng and T. Ma, CS229 Lecture Notes
- Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin, Learning From Data: A Short Course. New York, NY: AMLBook, 2012.